FAILSAFE

Early Student Failure Risk Detection System

Prem Kadam Bhavit Krishna Jain

Indian Institute of Technology Guwahati

This report documents the full implementation of FAILSAFE, a system that predicts which students are at risk of failing before the semester ends. The system uses machine learning to analyse attendance, prior grades, and behavioural data, and produces an explanation for every prediction along with a personalised intervention plan for each student. The following sections walk through the machine learning pipeline, model export, backend, frontend, and deployment.

1. Machine Learning Pipeline

The training notebook covers the complete pipeline from raw data to a deployable model. The dataset used is the UCI Student Performance Dataset, which has two CSV files: one for Mathematics students and one for Portuguese language students. Both files were merged into a single dataset of 1044 rows and 33 features. The dataset had no missing values.

Data loading and exploratory analysis

Both CSV files were uploaded to Colab and concatenated after removing any duplicate rows. We looked at the distribution of final grades (G3), the spread of absent days across students, and how prior failures relate to final performance. About 22 percent of students had a final grade below 10, meaning the dataset has a moderate class imbalance.

Feature engineering

We created a binary target column called at_risk. Any student with a final grade G3 below 10 was assigned at_risk = 1, and all others were assigned at_risk = 0. G3 was then dropped from the feature set. Keeping it would have caused direct target leakage since the label was derived from it. The period grades G1 and G2 were kept as features because they are collected before G3 and represent legitimate prior information.

Train and test split

The dataset was split into training and test sets (80/20) before any encoding was applied. This ordering is important. If encoders are fit on the full dataset before splitting, the test set's statistical distribution leaks into the training preprocessing, which inflates performance estimates. Stratification was used to ensure both splits had the same proportion of at-risk

students.

Preprocessing

Seventeen categorical features (school type, sex, address, family size, parental jobs, and others) were encoded using scikit-learn's LabelEncoder. The encoder was fit only on the training set. The test set was then transformed using the already-fitted encoder, never re-fit. This ensures no information about the test distribution enters the preprocessing step.

A scale_pos_weight value of 3.54 was computed from the class ratio (number of safe students divided by number of at-risk students). This is passed to XGBoost to handle the class imbalance.

Baseline XGBoost model

A baseline XGBoost classifier was trained with 100 estimators and default hyperparameters. It achieved a ROC-AUC of 0.96 on the test set, showing that the features carry strong predictive signal even without tuning.

Optuna hyperparameter tuning

We ran 50 trials with Optuna to find better hyperparameters. The parameters tuned were number of estimators, maximum tree depth, learning rate, row and column subsampling rates, L1 and L2 regularisation strength, and minimum child weight. Each trial was evaluated using 5-fold stratified cross-validation on the training set only. The test set was never seen during tuning. The best cross-validation F1 score was 0.8285.

Final model and overfitting check

The final model was trained using the best parameters found by Optuna. We compared train and test metrics side by side. The ROC-AUC gap between train and test was 0.03, which indicates the model generalises well. The F1 gap was slightly higher at 0.13, which is expected when the minority class has only 184 training samples. The final test ROC-AUC was 0.96 and accuracy was 89 percent.

SHAP explainability

SHAP (SHapley Additive exPlanations) was used to make every prediction interpretable. We used TreeExplainer, which is efficient for tree-based models and computes exact SHAP values rather than approximations. Three visualisations were generated: a global bar chart showing average feature importance across all students, a beeswarm plot showing the direction of each feature's impact, and per-student waterfall plots that break down exactly how each feature pushed that student's predicted risk up or down.

Intervention generator

After computing SHAP values for a student, the top risk-driving features are passed through a rule-based system. Each feature has an associated threshold (for example, more than 6 absences, G1 below 10, prior failures greater than or equal to 1) and a corresponding recommendation. The system returns a personalised list of actions for faculty to take before the semester ends.

2. Generating failsafe_model.pkl

After training, four objects were bundled together and saved using Python's pickle library: the trained XGBoost model, the dictionary of LabelEncoders keyed by column name, the ordered list of feature names, and the SHAP TreeExplainer. The feature name list preserves the exact column order the model was trained on. The encoders preserve the exact integer mapping for each categorical value. The SHAP explainer is saved because rebuilding it on every request would be slow. Together, this bundle is everything needed to run the full inference and explanation pipeline on new data without retraining.

3. Backend

The backend is a FastAPI application. When the server starts, it loads failsafe_model.pkl once into memory. All request handlers share the same loaded objects without reloading from disk on each request.

The main endpoint is POST /upload, which accepts a CSV file. The file is parsed using pandas (handling both semicolon and comma separators). G3 is dropped if present. Categorical columns are encoded using the saved LabelEncoders. The feature matrix is assembled in the exact column order stored in the bundle. XGBoost then produces a probability between 0 and 1 for each student, and students above 0.5 are classified as at-risk. SHAP values are computed for the entire batch and the intervention generator runs for each student. All results are stored in a SQLite database keyed by student ID.

Additional endpoints serve the frontend: GET /students returns all predictions sorted by risk score, GET /students/{id} returns the full SHAP values and interventions for a single student, and GET /summary returns aggregate statistics. The frontend HTML file is served directly from the FastAPI server at GET /app, so a single deployed server handles both the API and the interface.

4. Frontend

The frontend is a single HTML file. It loads React 18 and Tailwind CSS from CDN at runtime, so no build tools, npm, or Node.js are required. The entire interface is written as React components inside a Babel-transpiled script tag.

There are three views. The upload view presents a drag-and-drop area for CSV files. On submission, it calls the backend, waits for the response, and transitions to the dashboard. The dashboard shows four summary cards (total students, at-risk count, safe count, average risk score), a risk level distribution with three bars, and a searchable and filterable student table sorted by risk score. Clicking a row opens the student detail view, which shows the risk score, a SHAP feature contribution chart rendered as SVG, and the personalised intervention plan. For students who are not at risk, the section is titled Monitoring Recommendations rather than Recommended Interventions, and the content is still shown so there is no discrepancy between the table and the detail page.

5. Running Locally

Place failsafe_model.pkl in the same directory as main.py. Install dependencies with the command below, then start the server.

```
pip install -r requirements.txt
uvicorn main:app --reload
```

The server starts at http://localhost:8000. Uploading either student CSV file runs the full pipeline and opens the dashboard.

6. Deployment

Backend on Render

The repository includes a render.yaml file that specifies the build command (pip install -r requirements.txt) and start command (uvicorn main:app --host 0.0.0.0 --port $PORT). When the repository is connected to Render, this file is detected automatically and no manual configuration is needed. Every push to the main branch triggers a redeploy. The free tier provides 512 MB of RAM, which is sufficient to load the model and handle requests. The instance sleeps after periods of inactivity and takes around 30 seconds to wake on the first request, which is acceptable for a demo.

Frontend on Netlify

The frontend is deployed by dragging the single index.html file into Netlify's web interface. Before uploading, the API_BASE variable near the top of the file is updated to point to the Render backend URL. Netlify delivers the file over a global CDN. Anyone with the Netlify URL can open the dashboard, upload a CSV, and see predictions without installing anything.